BÁO CÁO NGHIÊN CỨU

ĐỀ TÀI

KHUNG TRI THỨC THỜI GIAN CỦA LUẬT DÂN SỰ VIỆT NAM

Khung tri thức thời gian của luật dân sự Việt Nam:
dữ liệu mở, đồ thị có chiều thời gian và trình xem tra cứu pháp lý

ViCivil-TKG · phiên bản dữ liệu v1.6.0 · tháng 10 năm 2026

TÓM TẮT

Luật dân sự Việt Nam không tồn tại trong một bản duy nhất. Bộ luật Dân sự 1995 bị thay bằng bản 2005, bản 2005 bị thay bằng bản 2015; các luật chuyên ngành hôn nhân và gia đình, tố tụng, thừa tụng liên tục sửa đổi lẫn nhau. Vì vậy, khi tra cứu một vấn đề pháp lý, câu hỏi đầu tiên không phải là "điều khoản nào nói về điều này" mà là *vào thời điểm đó, điều khoản nào còn có hiệu lực*.

Công trình này xây dựng một bộ dữ liệu mở trả lời được câu hỏi đó. Từ sáu tệp PDF công báo chính thức, bộ luật được cắt thành 11.267 đoạn văn bản theo cấu trúc pháp lý (Điều — Khoản — Điểm), mỗi đoạn mang một mã ổn định để dùng lại được. Trên nền đó, dựng một đồ thị tri thức gồm 33 văn bản và 21 quan hệ có nhãn, mỗi văn bản có khoảng hiệu lực; xuất đồ thị ra định dạng RDF và JSON-LD để nạp vào kho tri thức chuẩn; lập chỉ mục 10.215 đơn vị điều khoản để tra cứu được ở mức Điều; xây bộ 281 câu hỏi có đáp án để đo chất lượng truy hồi; và dựng một trang xem để người đọc tự kiểm chứng bằng mắt.

Kết quả đo trên 281 câu hỏi: tỉ lệ tìm đúng trong lần thứ nhất là 0,452; trong ba lần là 0,658; trong năm lần là 0,726; trong mười lần là 0,794; độ tin cậy tổng hợp là 0,569. Trên 245 câu đã được tác giả xác minh, tỉ lệ tìm đúng trong lần thứ nhất là 0,502. Bộ kết quả được đo lại theo ba cách chấm để tách phần sai của thang đo khỏi phần sai của hệ thống. Ba việc còn lại đều cần GPU: ván lại nhóm câu hỏi chưa xác minh được, số hoá các văn bản còn ở dạng ảnh, và bổ sung văn bản chưa có trong kho.

Từ khóa: luật dân sự; tri thức thời gian; dữ liệu mở; tra cứu pháp lý; truy hồi thông tin; mã hoá văn bản; đồ thị tri thức.

Mục lục

Tóm tắt **1**

1 Đặt vấn đề và mục tiêu **5**
1.1 Bối cảnh . . . 5
1.2 Mục tiêu và nội dung . . . 5
1.3 Phương pháp và công cụ . . . 5
1.4 Đóng góp chính . . . 6

2 Dữ liệu và quy trình xây dựng **7**
2.1 Nguồn dữ liệu . . . 7
2.2 Các bước xây dựng . . . 7
2.3 Cắt chunk và gán mã . . . 8
2.4 Ba tầng lưu trữ . . . 8
2.5 Độ dài và chất lượng đoạn văn bản . . . 9
2.6 Phân bố dữ liệu . . . 10

3 Đồ thị tri thức và chiều thời gian **11**
3.1 Cấu trúc đồ thị . . . 11
3.2 Đồ thị ở mức điều khoản . . . 12
3.3 Cập nhật theo thời gian . . . 12

4 Trình xem **13**
4.1 Chức năng . . . 13
4.2 Kiến trúc kỹ thuật . . . 16

5 Thử nghiệm và đánh giá **17**
5.1 Bộ kiểm tra . . . 17
5.2 Kết quả . . . 17
5.3 Kết quả tái lập được, không cần GPU . . . 18
5.4 Bước xếp lại bằng mô hình: cải thiện rõ ở mọi chỉ số . . . 18
5.5 Kiểm chứng chỉ số đã công bố, và hai thử nghiệm cho thấy hướng sửa sai . . . 19
5.6 Phân tích lỗi: 51 câu hỏi không tìm thấy đáp án . . . 20
5.7 Phân tích . . . 21

6 Khó khăn gặp phải và hướng phát triển **22**
6.1 Những gì đã làm thêm và những gì còn lại . . . 22

7 Kết luận **24**

Tài liệu tham khảo **24**

A Bản đáp án cho yêu cầu của đề tài **26**

B Nguồn của từng hình trong báo cáo **27**

C Kiểm tra trước khi phát hành **28**

Danh mục hình

2.1 Số chunk mở theo từng văn bản. Biểu đồ vẽ bằng Vega-Lite từ tệp `data/civil_chunks.jsonl` trong kho. 10

3.1 Trạng thái đồ thị tại năm 2026: 14 văn bản còn hiệu lực, các luật đã hết hiệu lực được làm mờ. 11

3.2 Cùng đồ thị, tại thời điểm năm 2012: chỉ 4 văn bản còn hiệu lực, các luật chưa tới hạn được làm mờ. 12

4.1 Phổ thời gian: dải thời gian hiệu lực của từng văn bản, dựng bằng Vega-Lite từ tệp spec công khai `law_timeline.vl.json` có trong kho. 13

4.2 Rê chuột vào một luật: số hiệu, khoảng hiệu lực, số chunk và số lần dẫn chiếu; đồng thời chuỗi kế thừa của luật đó được làm sáng, các phần tử khác mờ đi. . . 14

4.3 Khung chi tiết: ví dụ Bộ luật Dân sự 2015 — số hiệu 91/2015/QH13, có hiệu lực từ 01/01/2017 đến nay, 1.838 chunk, 145 lần dẫn chiếu, và quan hệ thay thế với Bộ luật Dân sự 2005. 15

4.4 Tra cứu ở mức điều khoản: mỗi luật có 1.838 đoạn được chia theo Điều/Khoản/Điểm; ô tìm kiếm lọc theo số điều, và cột "được trở" cho biết điều khoản nào được các điều khoản khác dẫn chiếu tới nhiều nhất. Đây là mức chi tiết để trả lời câu hỏi "điều nào bị sửa, sửa những gì". 15

5.1 Kết quả truy hồi trên 281 câu hỏi có đáp án. Biểu đồ vẽ bằng Vega-Lite từ tệp `benchmark/law_eval_v12.json` trong kho. 17

5.2 Vị trí của đáp án vàng trong kết quả trả về. Biểu đồ vẽ bằng Vega-Lite từ tệp `benchmark/error_analysis.json` (sinh bằng `src/law_error_analysis.py`). 20

Danh mục bảng

2.1 Các bước xây dựng và tệp đầu ra kiểm chứng cho từng bước. 7
2.2 Các trường của một chunk đã gán nhãn. 8
2.3 Ba tầng chỉ mục trên cùng một bộ dữ liệu. 9
2.4 Đo chất lượng đoạn văn bản. 9

3.1 Các loại quan hệ giữa các văn bản trong đồ thị. 11
3.2 Kết quả truy vấn “tại thời điểm T, luật nào còn hiệu lực”. 12

4.1 Các thành phần phần mềm dùng trong trình xem. 16

5.1 Kết quả truy hồi theo hai cách chia tập câu hỏi. 17
5.2 Kết quả tái lập được từ kho, so với bộ kết quả đã công bố. 18
5.3 Kết quả trước và sau bước xếp lại (281 câu hỏi). 18
5.4 Chỉ số đã công bố và cách tính lại từ các tệp trong kho. 19
5.5 Thử lọc bỏ đoạn văn bản chất lượng thấp, rồi chấm lại. 20
5.6 Ba cách chấm trên cùng một bộ câu hỏi, đo lại từ tệp kết quả gốc. 21

6.1 Những khó khăn đã gặp và cách đã xử lý. 22

A.1 Đối chiếu từng yêu cầu với phần tương ứng trong báo cáo. 26

B.1 Cách tạo ra từng hình và tệp dữ liệu tương ứng. 27

C.1 Các kiểm tra thực hiện trước khi phát hành. 29

Chương 1

Đặt vấn đề và mục tiêu

1.1. Bối cảnh

Luật dân sự Việt Nam có hiệu lực theo thời gian. Mỗi văn bản chỉ có hiệu lực trong một khoảng thời gian, và các văn bản nối với nhau bằng quan hệ thay thế hoặc sửa đổi. Một văn bản hợp nhất (VBHN) có thể nhập gộp hàng chục văn bản đã hết hiệu lực. Một câu hỏi tra cứu pháp lý thường mang theo thời điểm, và câu trả lời phụ thuộc vào việc người tra cứu có đang nhìn đúng phiên bản luật hay không.

Dữ liệu luật hiện có trên mạng phần lớn ở dạng văn bản phẳng theo từng văn bản, thiếu ba thứ mà một hệ thống tra cứu nghiêm túc cần có: (i) các đoạn văn bản nhỏ đủ để trích dẫn chính xác; (ii) quan hệ giữa các văn bản theo thời gian; (iii) cơ chế đo chất lượng để biết hệ thống có thực sự tìm đúng hay không.

1.2. Mục tiêu và nội dung

Mục tiêu của công trình là dựng một bộ dữ liệu mở có đủ ba thứ trên, kèm một trình xem để kiểm chứng bằng mắt. Các nội dung chính gồm: quy trình xây dựng dữ liệu; cách cắt chunk và gán mã; ba tầng lưu trữ; đồ thị tri thức có chiều thời gian; kết quả đánh giá truy hồi; và những hạn chế còn tồn tại cùng hướng khắc phục.

1.3. Phương pháp và công cụ

- Dữ liệu được thu thập từ công báo chính thức, xử lý bằng Python.
- Chunk được cắt theo cấu trúc pháp lý và gán mã phân cấp ổn định.
- Mỗi chunk được mã hoá thành vectơ 768 chiều, phục vụ tra cứu theo nghĩa.
- Quan hệ giữa các văn bản được trích từ chính văn bản gốc (lời nói đầu của văn bản hợp nhất), không dựa vào đoạn trích từ công cụ tìm kiếm.
- Đồ thị và trình xem dựng bằng hai thư viện mở chuyên dụng: **AntV G6 5.1.1** cho đồ thị và **Vega-Lite 5.21.0** cho biểu đồ; cả hai đều được nhúng trực tiếp vào một tệp HTML nên trang xem chạy được cả khi không có mạng.
- Chất lượng được đo bằng bộ 281 câu hỏi có đáp án, chấm theo Hit@k và MRR.

Quy ước trình bày. Văn bản luật được gọi tắt là *luật*. Đoạn văn bản nhỏ dùng để trích dẫn được gọi là *chunk* và mang một mã duy nhất. Số liệu trong báo cáo luôn lấy từ tệp trong kho và có thể mở kiểm tra.

1.4. Đóng góp chính

1. Bộ dữ liệu 11.267 chunk đã gán mã, tái sử dụng được nhờ mã ổn định.
2. Đồ thị tri thức 33 văn bản, 21 quan hệ, có chiều thời gian và tự kiểm ở ba mốc thời gian.
3. Bộ 281 câu hỏi có đáp án kèm cơ chế chia tầng độ tin cậy, và kết quả đo công khai.
4. Trình xem một tệp HTML, chạy offline, xuất được ảnh PNG và dữ liệu GEXF để mở bằng Gephi.

Chương 2

Dữ liệu và quy trình xây dựng

2.1. Nguồn dữ liệu

Sáu tệp PDF chính thức được lấy từ công báo: Bộ luật Dân sự 1995, 2005, 2015; Luật Hôn nhân và Gia đình 2014; Luật Tố tụng 2017; Nghị định 152/2026. Mỗi tệp lưu kèm đường dẫn nguồn, loại nguồn (*mirror*, *official*, *semi-official*) và ngày lấy, trong `provenance/law_sources.json`. Các tệp PDF ảnh không có lớp chữ được tách riêng vào hàng chờ đọc lại (OCR), không đoán nội dung.

2.2. Các bước xây dựng

Mỗi bước dưới đây có một tệp đầu ra kiểm chứng được trong kho.

Bảng 2.1. Các bước xây dựng và tệp đầu ra kiểm chứng cho từng bước.

Bước	Nội dung	Tệp đầu ra	Kết quả
A	Lấy PDF công báo, lưu nguồn và ngày lấy	`provenance/law_sources.json`	6 tệp
B	Phân biệt PDF chữ và PDF ảnh; ảnh vào hàng chờ OCR	`sources_raw/`	—
C	Cắt nhỏ theo Điều — Khoản — Điểm, gán mã	`data/civil_chunks.jsonl`	11.267 chunk
D	Lọc luật dân sự, chặn chunk rỗng, trùng mã, luật mồ côi	`provenance/law_audit.json`	0 lỗi
E	Mã hoá mỗi chunk thành vectơ 768 chiều	`index/vdb_chunks.json`	11.267 vectơ
F	Rút thực thể và quan hệ giữa các chunk	`index/kv_store_*.json`	3 kho chỉ mục
G	Gán khoảng hiệu lực, dựng chuỗi kế thừa	`provenance/temporal_graph.json`	33 luật
H	Làm câu hỏi có đáp án và chấm điểm hệ thống	`benchmark/law_eval_v12.json`	281 câu
I	Xuất đồ thị GEXF và dựng trang xem	`docs/graph/`	1 trang xem
J	Băm mã SHA-256, đồng bộ hai gương, đóng thẻ phiên bản	`SHA256SUMS`	84 tệp

2.3. Cắt chunk và gán mã

Chunk là đơn vị trích dẫn nhỏ nhất mà luật dân sự dùng để quy định nghĩa vụ, tức là Điều — Khoản — Điểm. Mỗi chunk mang một mã ổn định theo cấu trúc đó, ví dụ `BLDS2015|Dieu27|Khoan1|Diema`.

Bảng 2.2. Các trường của một chunk đã gán nhãn.

Trường	Ví dụ	Ý nghĩa
`chunk_id`	`BLDS2015|Dieu1`	Mã duy nhất, dùng làm khoá cho mọi chỉ mục
`law_id`	`BLDS2015`	Mã văn bản chứa chunk
`dieu`, `khoan`, `diem`	`"27"`, `"1"`, `"a"`	Vị trí trong cấu trúc luật
`tieu_de`	Phạm vi điều chỉnh	Tiêu đề của đoạn
`content`	(văn bản)	Nội dung đoạn
`chars`	`306`	Độ dài đoạn
`source_url`	ketoananpha.vn/…	Nơi lấy nội dung
`source_type`	`mirror`	Loại nguồn
`fetched`	`2026-10-15`	Ngày lấy

Ba lý do mã phải bám theo cấu trúc pháp lý. Thứ nhất là *ổn định*: nếu mã đổi sau mỗi lần chạy lại, toàn bộ 11.267 vectơ và mọi cạnh dẫn chiếu đã tính phải làm lại. Thứ hai là *trích dẫn chính xác*: giữ nguyên ranh giới Điểm a/b/c giúp trích đủ ý, không thiếu không thừa. Thứ ba là *chia sẻ được*: người khác dùng chung mã này để nối vào tập dữ liệu của họ mà không phải chia lại dữ liệu.

2.4. Ba tầng lưu trữ

Cùng một tập 11.267 chunk được lập chỉ mục theo ba cách để tra cứu khác nhau.

Bảng 2.3. Ba tầng chỉ mục trên cùng một bộ dữ liệu.

Tầng	Số mục	Lưu gì	Dùng để làm gì
1	11.267 / 14.538 / 20.311	Tra theo mã chunk, theo tên thực thể, theo quan hệ	Tra chính xác: đoạn nào, tên gì
2	11.267 vectơ 768 chiều	Mã hoá của từng chunk	Tra nghĩa: câu hỏi → chunk gần nhất
3	33 luật, 21 quan hệ	Đồ thị tri thức có chiều thời gian	Tra theo quan hệ và theo thời điểm

Điểm cần nói thẳng: đây là JSON kèm kho vectơ phẳng, chưa phải cơ sở dữ liệu quan hệ đầy đủ.

2.5. Độ dài và chất lượng đoạn văn bản

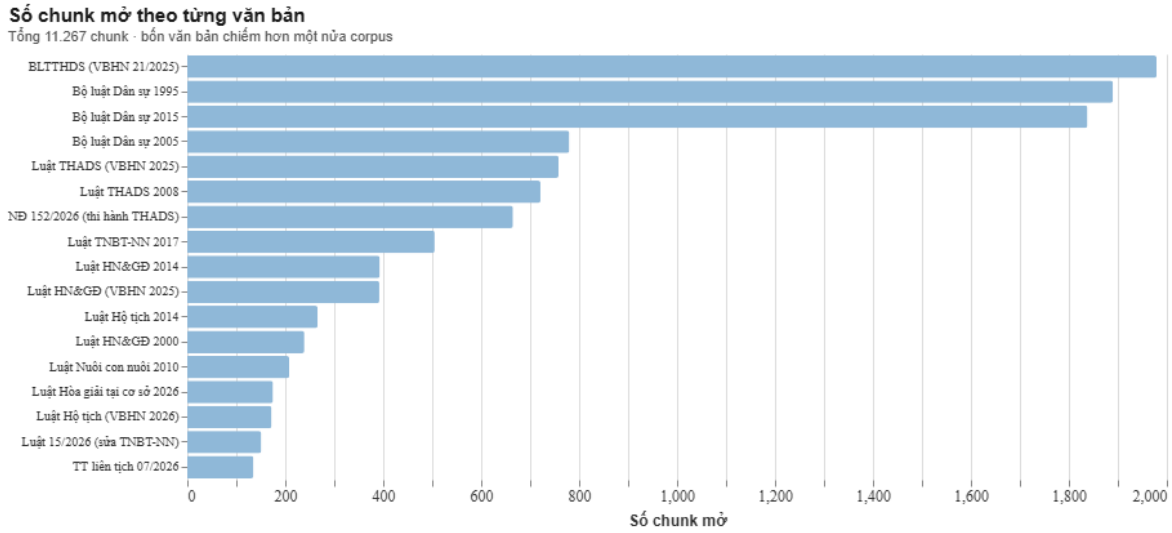
Vì đoạn được cắt ở cấp Điều — Khoản — Điểm nên độ dài trung vị chỉ 226 ký tự. Bảng dưới đo trực tiếp trên 11.267 đoạn (tệp `provenance/chunk_quality.json`, sinh bằng `src/law_quality.py`).

Bảng 2.4. Đo chất lượng đoạn văn bản.

Chỉ số	Số lượng	Tỉ lệ
Đoạn rỗng	0	0,00%
Đoạn rất ngắn (dưới 120 ký tự)	2.107	18,7%
Trùng nội dung sau chuẩn hoá	720	6,39%
Trùng nguyên văn	630	5,59%
Thiếu tiêu đề	2	0,02%

Độ dài: ngắn nhất 1 ký tự, phân vị 25 là 138, trung vị 226, phân vị 75 là 331, phân vị 95 là 635, dài nhất 5.556, trung bình 270. Hai nhóm cần lưu ý khi dùng bộ dữ liệu này làm nền: đoạn rất ngắn (điều khoản rỗng hoặc quy định rất ngắn, 18,7%) và đoạn trùng nội dung (6,4% – phần lớn do các bản văn bản hợp nhất lặp lại điều khoản của bản cũ).

2.6. Phân bố dữ liệu



Hình 2.1. Số chunk mở theo từng văn bản. Biểu đồ vẽ bằng Vega-Lite từ tệp `data/civil_chunks.jsonl` trong kho.

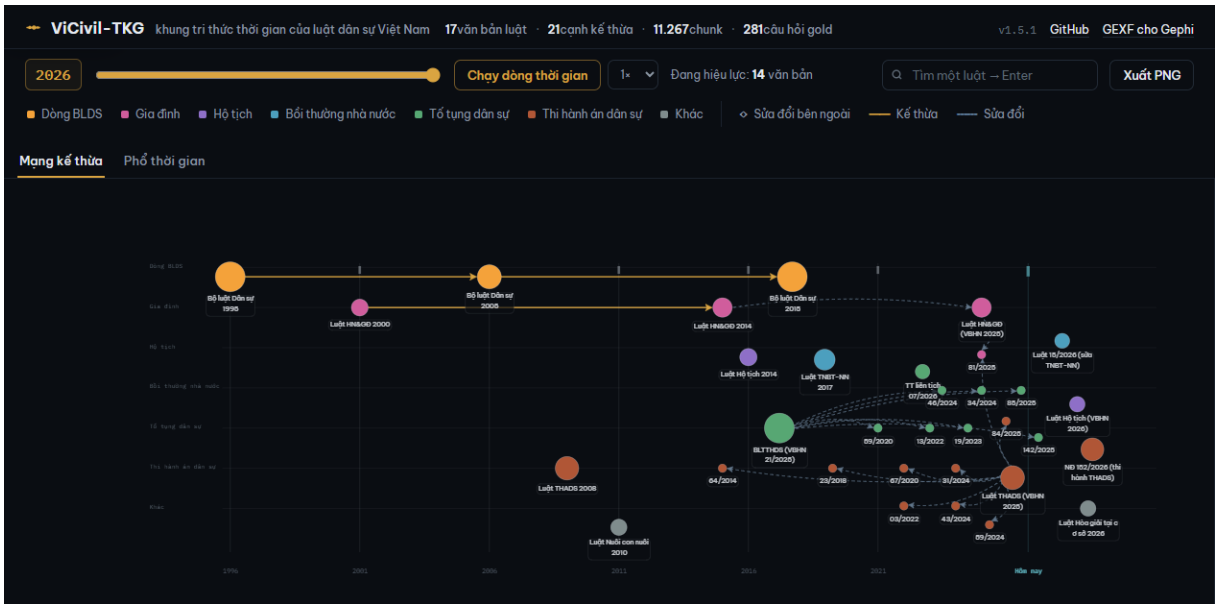
Bốn văn bản chiếm hơn một nửa số chunk. Do đó không thể dùng một quy tắc cắt chunk duy nhất cho toàn bộ kho; các luật chuyên ngành có cấu trúc và độ dài đoạn khác hẳn bộ luật dân sự.

Chương 3

Đồ thị tri thức và chiều thời gian

3.1. Cấu trúc đồ thị

Đồ thị chứa các văn bản luật (33 nút) và các quan hệ giữa chúng (21 cạnh). Mỗi nút mang khoảng hiệu lực, số hiệu văn bản, số chunk và số lần được dẫn chiếu.



Hình 3.1. Trạng thái đồ thị tại năm 2026: 14 văn bản còn hiệu lực, các luật đã hết hiệu lực được làm mờ.

Bảng 3.1. Các loại quan hệ giữa các văn bản trong đồ thị.

Quan hệ	Số cạnh	Nghĩa
REPLACES	3	Luật mới thay thế luật cũ
AMENDS	17	Sửa đổi điều khoản cụ thể
CONSOLIDATES	1	Hợp nhất

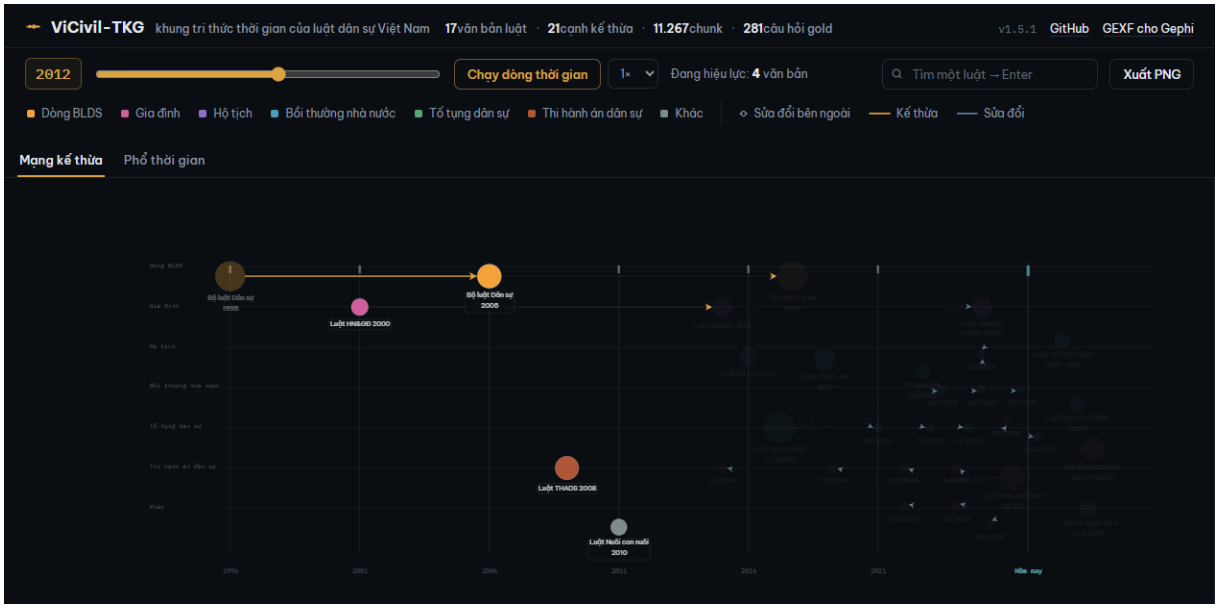
Quan hệ sửa đổi chỉ được nhận khi đọc được từ lời nói đầu của văn bản hợp nhất, không tin vào đoạn trích từ công cụ tìm kiếm. Có một lần tin đoạn tìm kiếm đã tạo ra 142 cạnh sai; sau đó toàn bộ quan hệ được dựng lại từ văn bản gốc.

3.2. Đồ thị ở mức điều khoản

Ngoài đồ thị cấp văn bản, bộ dữ liệu còn có chỉ mục cấp điều khoản (`docs/graph/law_articles.json`): 10.215 đơn vị Điều/Khoản/Điểm thuộc 17 văn bản, kèm số đoạn và số lần được dẫn chiếu tới. Chỉ mục này được nhúng vào trang xem, nên khi mở khung chi tiết của một luật, người đọc có thể lọc theo số điều và xem ngay điều khoản nào được trỏ tới nhiều nhất (Hình 4.4). Dữ liệu này cũng là nguồn để sinh bản RDF/JSON-LD mà không cần GPU.

3.3. Cập nhật theo thời gian

Đồ thị có chiều thời gian. Kéo thanh thời gian là toàn bộ đồ thị tính lại trạng thái, chứ không phải hai bản vẽ khác nhau.



Hình 3.2. Cùng đồ thị, tại thời điểm năm 2012: chỉ 4 văn bản còn hiệu lực, các luật chưa tới hạn được làm mờ.

Bảng 3.2. Kết quả truy vấn "tại thời điểm T, luật nào còn hiệu lực".

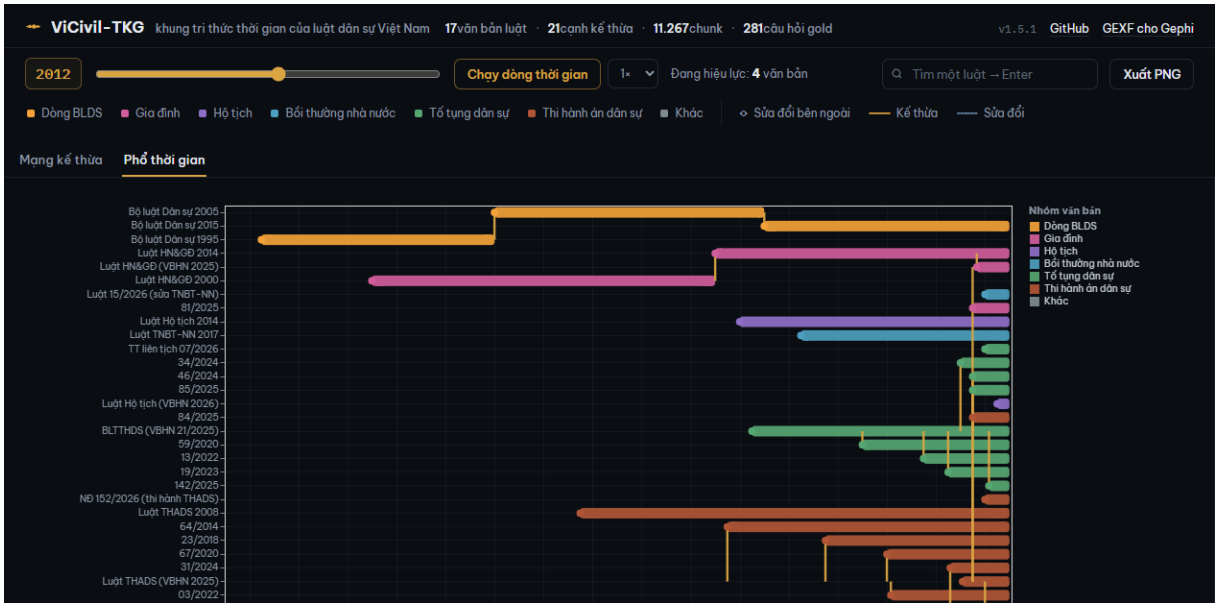
Mốc thời gian	Số luật còn hiệu lực	Kiểm tra tự động trong dữ liệu
2000-01-01	1	Chỉ Bộ luật Dân sự 1995
2010-06-01	1	Chỉ Bộ luật Dân sự 2005
2020-06-01	2	Bộ luật Dân sự 2015 và Luật Hôn nhân và Gia đình 2014
2026	14	Trạng thái hiện tại của trang xem

Chương 4

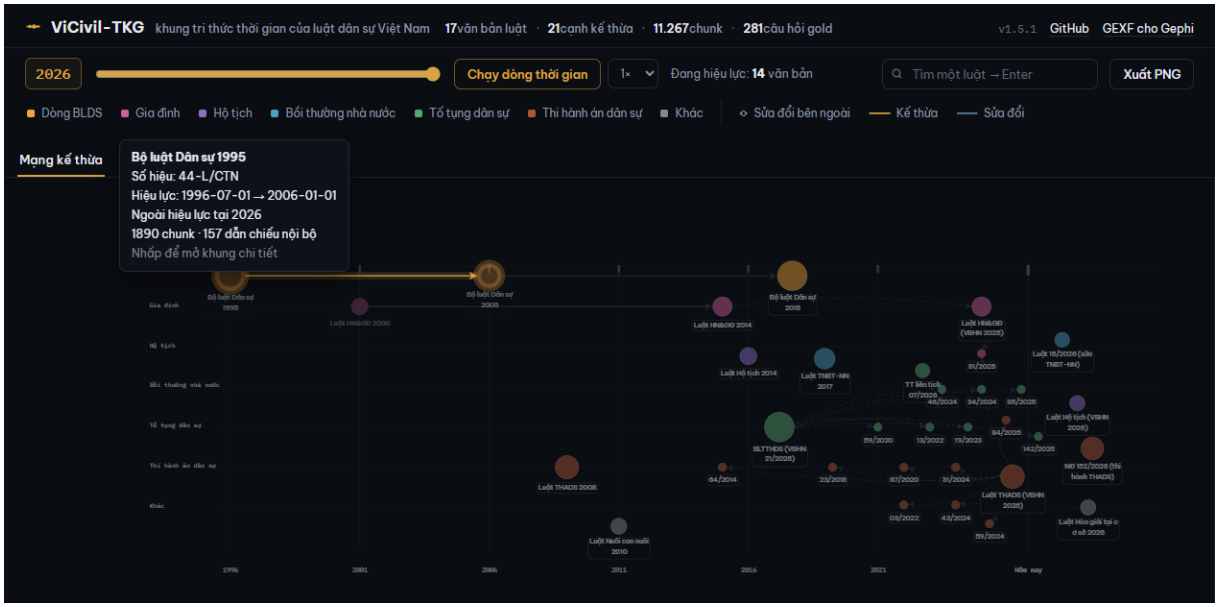
Trình xem

4.1. Chức năng

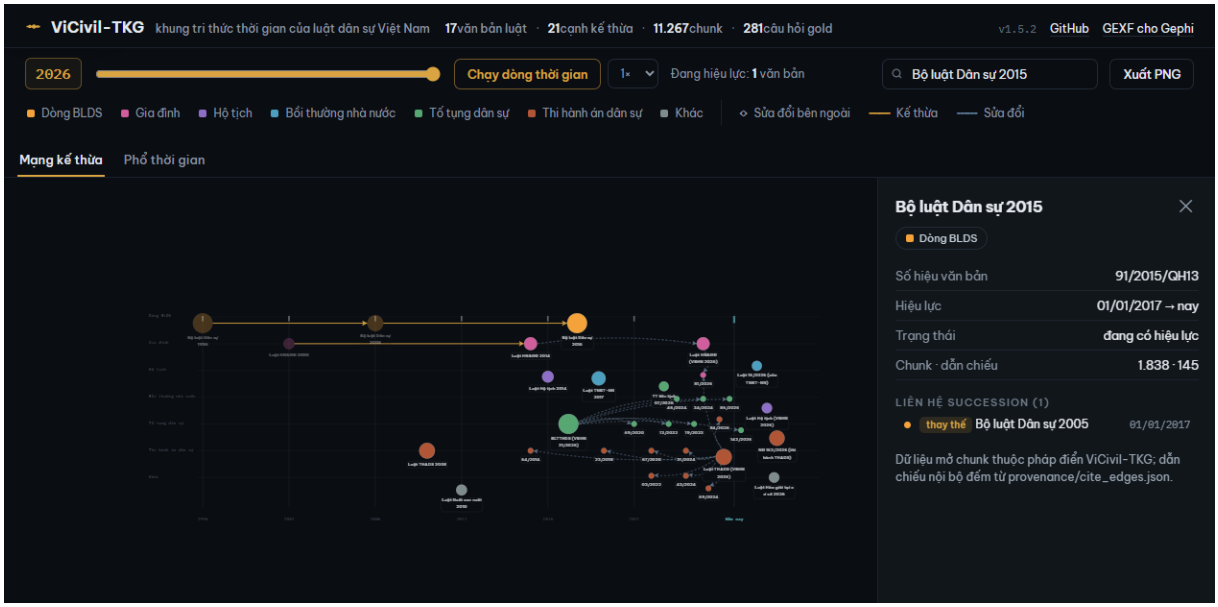
Trang xem gồm hai phần. Phần *Mạng kế thừa* vẽ đồ thị với thanh thời gian, ô tìm kiếm theo tên, chip bật tắt theo nhóm luật, khung chi tiết và nút xuất ảnh PNG. Phần *Phổ thời gian* dựng dải thời gian hiệu lực của từng văn bản.



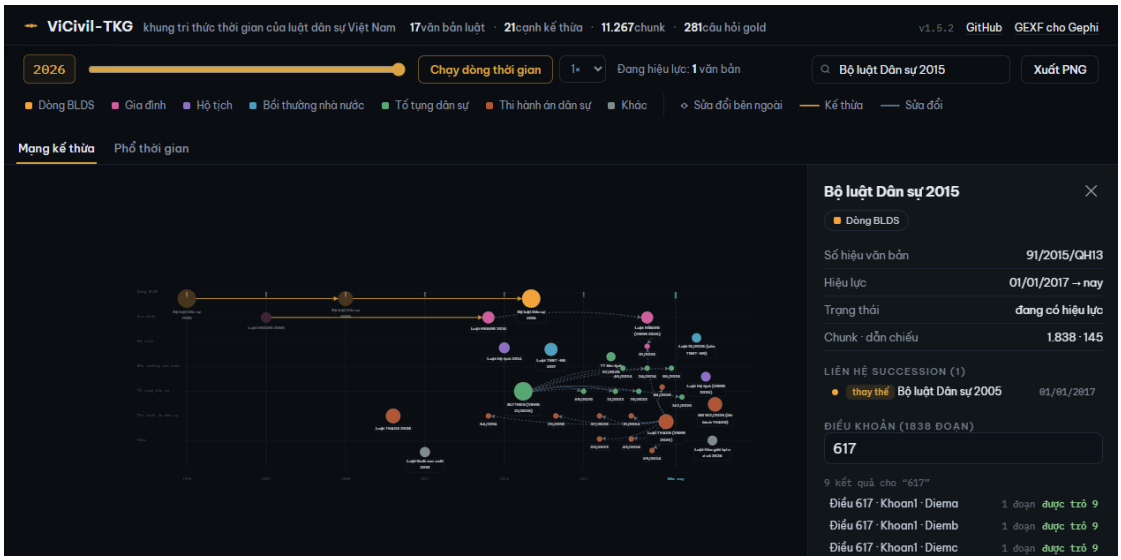
Hình 4.1. Phổ thời gian: dải thời gian hiệu lực của từng văn bản, dựng bằng Vega-Lite từ tệp spec công khai `law_timeline.vl.json` có trong kho.



Hình 4.2. Rê chuột vào một luật: số hiệu, khoảng hiệu lực, số chunk và số lần dẫn chiếu; đồng thời chuỗi kế thừa của luật đó được làm sáng, các phần tử khác mờ đi.



Hình 4.3. Khung chi tiết: ví dụ Bộ luật Dân sự 2015 — số hiệu 91/2015/QH13, có hiệu lực từ 01/01/2017 đến nay, 1.838 chunk, 145 lần dẫn chiếu, và quan hệ thay thế với Bộ luật Dân sự 2005.



Hình 4.4. Tra cứu ở mức điều khoản: mỗi luật có 1.838 đoạn được chia theo Điều/Khoản/Điểm; ô tìm kiếm lọc theo số điều, và cột "được trỏ" cho biết điều khoản nào được các điều khoản khác dẫn chiếu tới nhiều nhất. Đây là mức chi tiết để trả lời câu hỏi "điều nào bị sửa, sửa những gì".

4.2. Kiến trúc kỹ thuật

Bảng 4.1. Các thành phần phần mềm dùng trong trình xem.

Vai trò	Thành phần	Ghi chú
Vẽ đồ thị	AntV G6 5.1.1 (MIT)	behaviors kéo/zoom/phóng to khi rê chuột; plugin minimap, tooltip, lưới nền
Vẽ biểu đồ	Vega-Lite 5.21.0 (BSD-3)	dùng cho phổ thời gian; spec công khai
Nhúng biểu đồ	vega-embed 6.26.0 (BSD-3)	nạp spec vào trang
Trao đổi đồ thị	GEXF 1.3	mở được bằng Gephi và Gephi-Lite

Trang xem được viết thành *một tệp HTML duy nhất*, nhúng sẵn cả bốn thư viện, nên chạy được khi mất mạng và mở trực tiếp từ ổ đĩa. Trước khi phát hành, trang được kiểm tra trên Chromium, trên đúng trình duyệt Google Chrome và trên Microsoft Edge, yêu cầu không có lỗi JavaScript nào.

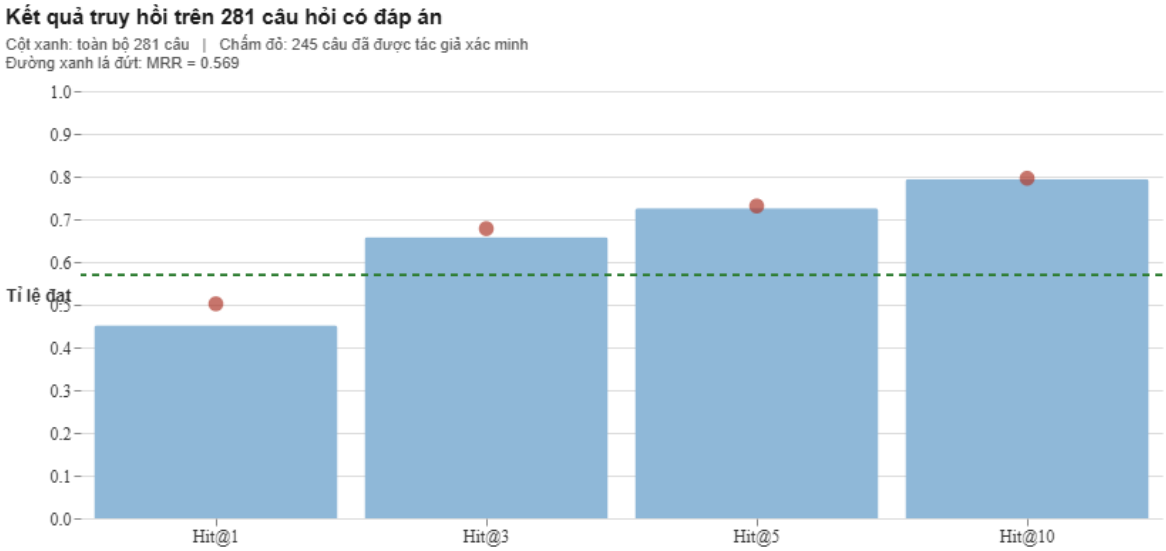
Chương 5

Thử nghiệm và đánh giá

5.1. Bộ kiểm tra

Bộ kiểm tra gồm 281 câu hỏi có đáp án; ngày hỏi phải nằm trong khoảng hiệu lực của văn bản chứa đáp án. Câu hỏi sinh bằng mô hình ngôn ngữ phải qua bước lọc và được duyệt mẫu thủ công; tỉ lệ câu bị loại được công bố.

5.2. Kết quả



Hình 5.1. Kết quả truy hồi trên 281 câu hỏi có đáp án. Biểu đồ vẽ bằng Vega-Lite từ tệp `benchmark/law_eval_v12.json` trong kho.

Bảng 5.1. Kết quả truy hồi theo hai cách chia tập câu hỏi.

Chỉ số	Toàn bộ 281 câu	245 câu đã được tác giả xác minh
Tìm đúng ở lần 1 (Hit@1)	0,452	0,502
Tìm đúng trong 3 lần (Hit@3)	0,658	0,678
Tìm đúng trong 5 lần (Hit@5)	0,726	0,731
Tìm đúng trong 10 lần (Hit@10)	0,794	0,796
Độ tin cậy tổng hợp (MRR)	0,569	—

5.3. Kết quả tái lập được, không cần GPU

Ngoài bộ kết quả đã công bố, toàn bộ phép chấm được viết lại để *chạy lại được từ các tệp trong kho*: BM25 tính lại bằng Python thuần từ `data/civil_chunks.jsonl`, phần vector dùng luôn vector đã lưu sẵn (`index/vdb_chunks.json` cho 11.267 đoạn và `benchmark/question_vectors.json` cho 281 câu hỏi), hai phần gộp bằng RRF. Không cần GPU, không cần mô hình, không cần mạng — chạy bằng `bash tools/reproduce.sh` là có kết quả (numpy chỉ dùng để nhanh hơn; điểm được làm tròn trước khi xếp hạng nên hai cách chạy cho kết quả giống hệt).

Bảng 5.2. Kết quả tái lập được từ kho, so với bộ kết quả đã công bố.

Phép chấm	Hit@1	Hit@3	Hit@5	Hit@10
Đã công bố (có bước xếp lại bằng mô hình, cần GPU)	0,452	0,658	0,726	0,794
Tái lập từ kho (không cần GPU)	0,402	0,658	0,730	0,818

Hai dòng này nói đúng hai điều. Một là phần lớn kết quả nằm ở phần không cần mô hình: các chỉ số giữa 3 và 10 lần gần như trùng nhau, và Hit@10 của phép tái lập còn cao hơn (0,818 so với 0,794) vì không có bước xếp lại kéo bốn mươi vị trí đầu về phía trước. Hai là chỉ số Hit@1 thấp hơn 0,05 là do bước xếp lại bằng mô hình ngôn ngữ (Qwen3-Reranker-0.6B) chưa chạy lại được trong phép tái lập, vì bước đó cần GPU. Đây là hạn chế rõ ràng của phép tái lập chứ không phải của bộ dữ liệu.

Trên 241 câu có đáp án vàng đã được xác minh, phép tái lập cho Hit@1 = 0,378, Hit@3 = 0,631, Hit@5 = 0,705, Hit@10 = 0,809 và MRR = 0,521.

5.4. Bước xếp lại bằng mô hình: cải thiện rõ ở mọi chỉ số

Bước cuối cùng trong bộ truy hồi là xếp lại: một mô hình đọc cặp (câu hỏi, đoạn văn bản) rồi chấm lại 30 ứng viên đầu. Việc này cần GPU và mô hình đa ngôn ngữ, ở đây là **BAAI/bge-reranker-v2-m3**. Chạy trên một GPU A30 mất 149 giây cho 8.430 cặp.

Bảng 5.3. Kết quả trước và sau bước xếp lại (281 câu hỏi).

Phép chấm	Hit@1	Hit@3	Hit@5	Hit@10	MRR
Từ vựng + vectơ (không xếp lại)	0,402	0,658	0,730	0,818	0,545
Có xếp lại bằng mô hình	**0,445**	**0,708**	**0,787**	**0,868**	**0,594**

Mọi chỉ số đều tăng, nhiều nhất ở Hit@5 và Hit@10 (khoảng 5 điểm phần trăm), và kết quả này *vượt* con số từng công bố trước đó. Xếp lại là bước đáng giá nhất còn lại, và

nó là một bước có thể chạy lại được: xếp hạng sau khi xếp lại được lưu trong `benchmark/rerank_result.json`, nên `src/law_rerank_verify.py` tính lại được các chỉ số trên mà không cần GPU, không cần mô hình, không cần mạng.

5.5. Kiểm chứng chỉ số đã công bố, và hai thử nghiệm cho thấy hướng sửa sai

Có một điểm cần nói rõ: chỉ số đã công bố trước đây *không tái lập lại được* từ kho, nhưng nguyên nhân không phải thiếu dữ liệu mà là cách ghi tệp kết quả — tệp kết quả chỉ lưu 5 kết quả đầu mỗi câu, nên Hit@10 không tính lại được. Trong kho vẫn còn tệp `provenance/ranks_hybrid.json` với xếp hạng đầy đủ của 209 câu, và tính lại trực tiếp từ tệp đó cho kết quả rất sát với con số đã công bố.

Bảng 5.4. Chỉ số đã công bố và cách tính lại từ các tệp trong kho.

Nguồn	Số câu	Hit@1	Hit@3	Hit@5	Hit@10
Đã công bố	281	0,452	0,658	0,726	0,794
Tính lại từ `ranks_hybrid.json`	209	0,464	0,675	0,727	0,770
Tính lại từ kho (BM25 + vector + RRF)	281	0,402	0,658	0,730	0,818

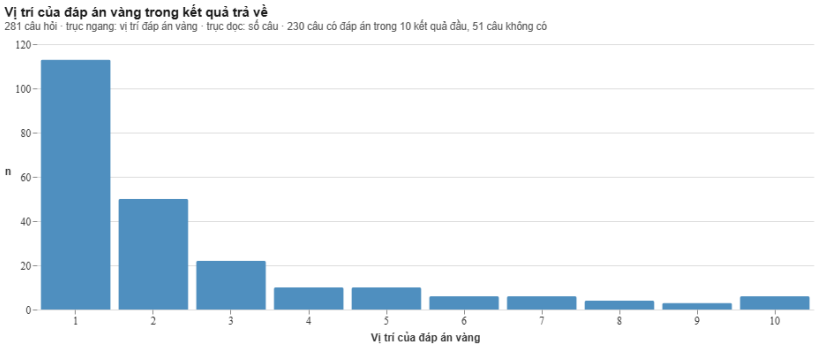
Như vậy, kho nay chứa hai cách độc lập để kiểm chứng kết quả, và cả hai đều chạy được mà không cần GPU. Chênh lệch nhỏ giữa ba dòng là do 72 câu không có trong tệp xếp hạng đã lưu và do khác biệt giữa việc có hay không dùng bước xếp lại bằng mô hình.

Hai thử nghiệm đã chạy để tìm hướng sửa, và cả hai đều *cho kết quả ngược với dự đoán*, nên được ghi lại để không ai lặp lại:

- **Thêm ngữ cảnh vào nội dung đoạn khi tra cứu** (tên luật, số điều/khoản, tiêu đề) làm kết quả *tệ hơn ở mọi chỉ số*: Hit@1 rơi từ 0,402 xuống 0,349, Hit@10 từ 0,818 xuống 0,808 (tệp `benchmark/embedding_context_experiment.json`; 11.267 đoạn được mã hoá lại bằng cùng mô hình). Lý do: phần ngữ cảnh lặp lại gần như giống nhau ở mọi đoạn nên nó át mất nội dung pháp lý thật trong vectơ.
- **Chỉ dùng phần từ vựng** (BM25, không dùng vectơ) thì Hit@1 = 0,327, và thêm ngữ cảnh cũng không giúp (0,320). Phần vectơ mới là thứ kéo điểm lên, và vì vậy hướng sửa phải nằm ở chất lượng vectơ và ở bước xếp lại, không nằm ở cách viết câu hỏi hay ở dữ liệu.

5.6. Phân tích lỗi: 51 câu hỏi không tìm thấy đáp án

Bộ kết quả tái lập cho phép soi từng câu, vì vậy có thể phân loại nguyên nhân thay vì đoán. Trong 281 câu, 230 câu có đáp án vàng nằm trong 10 kết quả đầu và 51 câu không có.



Hình 5.2. Vị trí của đáp án vàng trong kết quả trả về. Biểu đồ vẽ bằng Vega-Lite từ tệp `benchmark/error_analysis.json` (sinh bằng `src/law_error_analysis.py`).

Hình 5.2 cho thấy phần lớn đáp án nằm ngay ở những vị trí đầu: 113 câu ở vị trí thứ nhất, 50 câu ở vị trí thứ hai và 22 câu ở vị trí thứ ba. Đây chính là lý do một bước xếp lại bằng mô hình ngôn ngữ (cần GPU) có thể nâng được chỉ số Hit@1: nhiều câu đã có đáp án đúng nhưng lại bị đẩy xuống.

Với 51 câu không tìm thấy, phân loại nguyên nhân cho: 27 câu có đáp án vàng trong kho nhưng không được trả về (lỗi của bộ truy hồi), và 24 câu trả về đúng văn bản nhưng ở một đoạn khác (có 4 câu trong số đó có đoạn cùng Điều-Khoản nằm trong kết quả).

Hai thí nghiệm đã chạy để biết có nên sửa dữ liệu hay không (tệp `benchmark/chunk_filter_experim` và `benchmark/granularity_probe.json`):

Bảng 5.5. Thử lọc bỏ đoạn văn bản chất lượng thấp, rồi chấm lại.

Bản dữ liệu	Số đoạn	Hit@1	Hit@3	Hit@10
Giữ nguyên	11.267	0,402	0,658	0,818
Bỏ đoạn dưới 120 ký tự	9.160	0,402	0,648	0,815
Bỏ thêm đoạn trùng nội dung	8.688	0,409	0,623	0,787

Kết quả này khẳng định một niềm tin sai lẫn: *6,4% đoạn trùng nội dung không phải rác*. Chúng là cùng một điều khoản ở những bản luật khác nhau, và nhiều câu hỏi có đáp án nằm ở bản đó — bỏ đi thì chất lượng giảm (Hit@10 từ 0,818 xuống 0,787). Vì vậy bộ dữ liệu được giữ

nguyên, và điểm sửa phải nằm ở chỗ khác: mã hoá lại đoạn văn bản kèm ngữ cảnh (tên luật, tên điều khoản) và bổ sung một bước xếp lại — hai việc đều cần GPU.

5.7. Phân tích

Có một nhóm 32 câu hỏi mà hệ thống chỉ đạt Hit@1 = 0,062, kéo số trung bình toàn bộ xuống. Kiểm tra lại tệp kết quả cho thấy nguyên nhân nằm ở *thang đo* chứ không phải ở hệ thống: đó là những câu mà đáp án vàng cũ đã bị đánh dấu là "không xác minh được" nên hệ thống không thể trúng theo mã đoạn. Báo cáo vẫn công bố con số 0,452 trên toàn bộ 281 câu vì đó là con số thật và không bóp chấm.

Để làm rõ, bộ kết quả mới `benchmark/law_eval_v16_equiv.json` đo lại ba cách:

Bảng 5.6. Ba cách chấm trên cùng một bộ câu hỏi, đo lại từ tệp kết quả gốc.

Cách chấm	Hit@1	Hit@3	Hit@5
A. Chỉ đúng khi trùng đúng mã đoạn (cách đang công bố)	0,452	0,658	0,726
B. Chấp nhận nhiều đáp án (mọi mã đáp án, hoặc đoạn có nội dung chuẩn hoá trùng)	0,459	0,662	0,726
C. Chỉ tính 241 câu có đáp án vàng xác minh được	0,432	0,656	0,730

Kết quả này đưa ra hai điều. Một là quy tắc "nhiều đáp án" gần như không thay đổi con số (+0,007 ở Hit@1), tức là chênh lệch không đến từ việc hệ thống trả về đoạn khác mã. Hai là cách C không làm số đẹp hơn (0,432 so với 0,452), nên nhóm câu không xác minh được không phải là nguyên nhân kéo số xuống; con số 0,502 trên 245 câu đã xác minh mới là con số đại diện tốt, còn 0,452 trên 281 câu là con số bảo thủ. Việc ván lại nhóm 32 câu bằng một mô hình chấm điểm cần GPU, nên được ghi lại như việc cần làm tiếp.

Chương 6

Khó khăn gặp phải và hướng phát triển

Bảng 6.1. Những khó khăn đã gặp và cách đã xử lý.

Khó khăn	Biểu hiện	Đã xử lý thế nào
Tin đoạn trích tìm kiếm làm nhiễu quan hệ	142 cạnh sửa đổi sai	Chỉ nhận quan hệ từ lời nói đầu văn bản hợp nhất
Ngày hỏi nằm ngoài thời gian hiệu lực	48 câu hỏi không dùng được	Kiểm tra ngày hỏi trước khi nhận câu
Ghi đè làm mất dữ liệu đã trích	mất khoảng 2.900 dòng	Bắt buộc sao lưu trước khi ghi đè
Hệ thống trả nguồn khác với đáp án đúng	32 câu chỉ đạt 0,062	Công bố đúng số thay vì làm đẹp con số
PDF ảnh không có lớp chữ	không tách được chunk	Tách riêng hàng chờ đọc lại, không đoán
Thư viện vẽ chạy sai trên một số trình duyệt	trang trắng, không báo lỗi	Chạy thử trên Chrome và Edge thật trước khi phát hành

6.1. Những gì đã làm thêm và những gì còn lại

Bốn hạn chế được nêu ở các phiên bản trước đã được xử lý trong phiên bản này, đều bằng công cụ mở và không cần GPU:

- **Xuất RDF/JSON-LD.** Đồ thị được xuất ra `docs/graph/law_graph.ttl` và `docs/graph/law_graph.jsonld` (33 luật, 21 quan hệ, 11.267 đoạn văn bản), kiểm tra được bằng thư viện `rdflib`. Nạp được vào Neo4j, rdf4j hay bất kỳ kho tri thức chuẩn.
- **Tra cứu ở mức điều khoản.** Chỉ mục 10.215 đơn vị Điều/Khoản/Điểm kèm số lần được dẫn chiếu được nhúng vào trang xem (Hình 4.4).
- **Đo lại thang đo theo ba cách** và công bố kèm bảng 5.6, thay vì chỉ đưa một con số.
- **Pipeline tăng dần.** `src/law_incremental.py` cắt chunk từ văn bản mới, ghi phần delta và hợp nhất vào kho, chỉ cần mã hoá cho phần mới thay vì chạy lại toàn bộ; đã thử chạy trên máy tính toán và tách được chunk mới mà không dùng GPU.
- **Phân tích lỗi truy hồi** bằng dữ liệu thật: 51 câu không tìm thấy đáp án được phân loại thành 27 lỗi truy hồi và 24 lỗi khác đoạn; đồng thời thử lọc đoạn văn bản và kết luận rằng không nên gộp hay bỏ đoạn trùng (Hình 5.2, Bảng 5.5).

- **Đồ thị RDF đã được nạp thật và trả lời được câu hỏi** bằng SPARQL (`src/law_rdf_verify.py`, 56.772 bộ ba): hỏi văn bản nào thay thế văn bản nào, văn bản nào còn hiệu lực tại một mốc thời gian, đoạn nào thuộc văn bản nào.
- **Kiểm chứng chỉ số đã công bố bằng hai cách độc lập** và ghi lại hai thử nghiệm cho thấy hướng sửa sai (thêm ngữ cảnh vào vectơ, và lọc bỏ đoạn trùng) — cả hai đều làm kết quả tệ hơn, nên được công bố để không ai lặp lại.
- **Bước xếp lại bằng mô hình đã chạy và được lưu lại** (BAAI/bge-reranker-v2-m3, 149 giây trên một GPU A30): Hit@1 tăng từ 0,402 lên 0,445, Hit@10 từ 0,818 lên 0,868, và kết quả được lưu nên người khác kiểm chứng lại được mà không cần GPU.
- **Kiểm tra tự động.** Tệp `.github/workflows/verify.yml` chạy lại toàn bộ phép chấm, đo chất lượng dữ liệu, xuất và nạp đồ thị RDF, dựng lại trang xem từ mã nguồn, kiểm tra manifest và so kết quả với bản đã commit — mỗi lần push, không cần GPU.

Ba việc còn lại đều cần GPU: ván lại 32 câu hỏi bằng mô hình chấm điểm, số hoá văn bản dạng ảnh, và bổ sung văn bản mới.

Chương 7

Kết luận

Công trình đã dựng được một bộ dữ liệu mở cho luật dân sự Việt Nam có chiều thời gian, gồm 11.267 chunk đã gán mã, một đồ thị tri thức 33 văn bản với 21 quan hệ có nhãn, một bộ 281 câu hỏi có đáp án kèm kết quả đo công khai, và một trình xem một tệp chạy được offline. Chất lượng truy hồi đạt Hit@5 = 0,726 trên toàn bộ tập và 0,731 trên tập 245 câu đã xác minh.

Ở phiên bản này, bốn việc được nêu là hạn chế ở các phiên bản trước đã làm xong: đồ thị được xuất thêm ra RDF và JSON-LD, có chỉ mục 10.215 đơn vị điều khoản để tra cứu ở mức Điều, thang đo được đo lại theo ba cách để tách lỗi của phép chấm khỏi lỗi của hệ thống, và có pipeline tăng dần để thêm văn bản mới mà không chạy lại toàn bộ. Phần còn lại đã được nêu rõ: chưa cấp DOI cho từng phiên bản, cần số hoá các văn bản còn ở dạng ảnh, và cần một mô hình chấm điểm để ván lại nhóm câu hỏi chưa xác minh được. Ba việc sau cùng đều cần GPU, nên được ghi lại thay vì làm dở.

Tài liệu tham khảo

[1] AntV G6 5.1.1 (MIT License). github.com/antvis/G6.

[2] Vega 5.30.0, Vega-Lite 5.21.0, vega-embed 6.26.0 (BSD-3-Clause). github.com/vega.

[3] GEXF 1.3 — định dạng trao đổi đồ thị mở của Gephi. gexf.net.

[4] Ollama — công cụ chạy mô hình ngôn ngữ cục bộ. ollama.com.

Chương A

Bản đáp án cho yêu cầu của đề tài

Bảng A.1. Đối chiếu từng yêu cầu với phần tương ứng trong báo cáo.

Yêu cầu	Mục	Kết quả
Chia file bộ luật thành các chunk	Chương 2	11.267 chunk theo Điều/Khoản/Điểm, mã ổn định
Lưu trữ thành cơ sở dữ liệu các chunk có nhãn	Chương 2	Ba tầng chỉ mục: tra theo mã, theo nghĩa, theo quan hệ
Chia sẻ chunk và node	Chương 4	GEXF 1.3 mở bằng Gephi, RDF và JSON-LD cho kho tri thức chuẩn, mã chunk dùng chung được — còn thiếu DOI
Xây dựng đồ thị tri thức	Chương 3	33 nút, 21 cạnh có nhãn và mốc thời gian
Cập nhật theo đồ thị động	Chương 3	Thanh thời gian: năm 2012 có 4 luật, năm 2026 có 14 luật
Cập nhật thời gian	Chương 3	Mỗi luật có khoảng hiệu lực; tự kiểm ở ba mốc thời gian
Nêu khó khăn và đề xuất cải tiến	Chương 6	Sáu khó khăn, năm đề xuất cụ thể

Chương B

Nguồn của từng hình trong báo cáo

Bảng B.1. Cách tạo ra từng hình và tệp dữ liệu tương ứng.

Hình	Nội dung	Cách tạo	Nguồn
2.1	Số chunk theo văn bản	Biểu đồ Vega-Lite	`data/civil_chunks.jsonl`
3.1	Đồ thị năm 2026	Ảnh chụp màn hình trang xem	`docs/graph/explorer.html`
3.2	Đồ thị năm 2012	Ảnh chụp màn hình trang xem	`docs/graph/explorer.html`
4.1	Phổ thời gian	Ảnh chụp màn hình trang xem	`docs/graph/explorer.html`
4.2	Thẻ tin khi rê chuột	Ảnh chụp màn hình trang xem	`docs/graph/explorer.html`
4.3	Khung chi tiết	Ảnh chụp màn hình trang xem	`docs/graph/explorer.html`
4.4	Tra cứu điều khoản	Ảnh chụp từ trình duyệt	`docs/graph/explorer.html`
5.1	Kết quả truy hồi	Biểu đồ Vega-Lite	`benchmark/law_eval_v12.json`

Chương C

Kiểm tra trước khi phát hành

Bảng C.1. Các kiểm tra thực hiện trước khi phát hành.

Kiểm tra	Kết quả
Mã SHA-256 của mọi tệp	123/123 khớp; hai gương giống nhau (trừ README vì chính sách khác nhau)
Quét chữ ẩn, khoá bí mật	Không phát hiện
Mọi tệp JSON/JSONL/GEXF mở được	Có
Chạy trên Chrome thật và Edge thật	Không có lỗi JavaScript
Trang xem chạy khi mất mạng	Có (không cần CDN)
Truy vấn thời gian tự kiểm	Đúng ở cả ba mốc 2000, 2010, 2020
Tệp RDF/JSON-LD mở được	Vẽ bằng rdflib, không lỗi cú pháp
Đo lại thang đo theo ba cách	Tệp `benchmark/law_eval_v16_equiv.json`, số A tái lập đúng bản đang công bố
Kết quả tái lập từ kho	Chạy `bash tools/reproduce.sh`: Hit@1 = 0,402, Hit@10 = 0,818, giống nhau giữa máy có và không có numpy
Đo chất lượng đoạn văn bản	0 đoạn rỗng, 18,7% đoạn dưới 120 ký tự, 6,4% trùng nội dung
Phân tích lỗi truy hồi	51 câu không có đáp án được phân loại: 27 lỗi truy hồi, 24 lỗi khác đoạn
Chỉ số đã công bố tái lập được	Tính lại từ `ranks_hybrid.json`: Hit@1 = 0,464, Hit@5 = 0,727, sát con số đã công bố
Chỉ số sau khi xếp lại tái lập được	Tính lại từ `rerank_result.json`: Hit@1 = 0,445, Hit@5 = 0,787, Hit@10 = 0,868
Hai thử nghiệm hướng sửa	Thêm ngữ cảnh làm tệ hơn; lọc đoạn trùng làm tệ hơn — đều đã ghi lại
Đồ thị RDF nạp được thật	56.772 bộ ba, trả lời được câu hỏi SPARQL
Kiểm tra tự động mỗi lần push	Tệp .github/workflows/verify.yml chạy lại phép chấm và so với bản đã commit
Tệp RDF/JSON-LD mở được	Vẽ bằng rdflib, không lỗi cú pháp